

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	31 January 2025
Team ID	
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Technical Architecture:

The deliverable includes the architectural diagram below and the information in Table-1 and Table-2.

Technical architecture of AI FAQ Assistant API

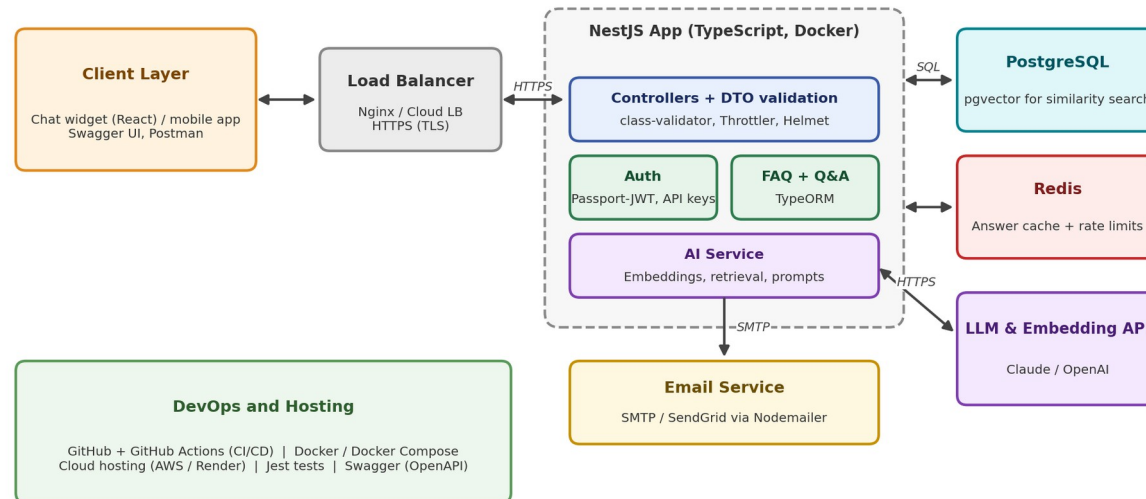


Figure 1: Technical architecture of AI FAQ Assistant API

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	How clients use the API: a website chat widget, mobile app or admin dashboard calls the REST endpoints; developers explore them through interactive documentation.	React chat widget (consumer), Swagger UI (OpenAPI), Postman
2	Application Logic-1	Account and access: registration, login, JWT access and refresh tokens, roles, API keys.	NestJS (TypeScript), Passport-JWT, bcrypt
3	Application Logic-2	FAQ management: CRUD, status workflow, categories, tags, search, CSV / JSON import.	NestJS modules, TypeORM, csv-parse
4	Application Logic-3	Q&A engine and AI service: embeddings, top-k retrieval, grounded prompt, confidence score, sessions and follow-ups.	NestJS provider services, @nestjs/axios (HTTP client), prompt templates
5	Application Logic-4	Feedback, escalation tickets, agent queue and analytics reports.	NestJS, TypeORM, Nodemailer
6	Database	Relational data plus embedding vectors: users, API keys, FAQs, conversations, feedback and tickets, with a vector index for similarity search.	PostgreSQL, pgvector extension, TypeORM migrations
7	Cloud Database	Managed PostgreSQL that supports pgvector, with automatic backups.	AWS RDS, Neon or Supabase
8	Cache	Stores cached answers and rate-limit counters.	Redis (cache-manager, @nestjs/throttler storage)
9	File Storage	Temporary storage for uploaded CSV / JSON import files. Local folder in development; object storage in production.	Local filesystem (dev), AWS S3 or compatible storage (production)
10	External API-1	Large language model that writes the final answer from the matched FAQs.	Claude API or OpenAI API (set by configuration)
11	External API-2	Embedding model that turns FAQs and questions into vectors.	Embedding API from the chosen provider (for example OpenAI embeddings)
12	External API-3	Sends confirmation and escalation emails.	SMTP or SendGrid through Nodemailer
13	Machine Learning Model	Hosted embedding and language models used through APIs for search and answer writing. No model is trained or hosted by the project.	Claude / GPT family models, text embedding models
14	Infrastructure (Server / Cloud)	Application deployment. Local: Docker Compose runs the API, PostgreSQL and Redis. Cloud: containers behind a load balancer, built and released through a CI/CD pipeline.	Docker, Docker Compose, GitHub Actions, AWS (ECS / EC2) or Render

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source frameworks and libraries used by the API.	NestJS, TypeORM, Passport, class-validator, pgvector, Jest, @nestjs/swagger
2	Security Implementations	Password hashing, JWT for staff, hashed API keys for client apps, role-based access control, strict organisation isolation, input validation, secure headers, CORS policy, rate limiting, HTTPS, secrets in environment variables, and prompt-injection safeguards, following OWASP Top 10 guidance.	bcrypt, JWT, Nest guards, Helmet, @nestjs/throttler, TLS
3	Scalable Architecture	Modular 3-tier design (clients, NestJS application, data layer). Stateless instances scale horizontally; embedding and import jobs run in a queue; the AI module can later become a separate microservice.	NestJS modules, Docker, job queue (BullMQ)
4	Availability	Load balancer over several containers, health checks, automatic restarts, managed database with backups, and graceful handling of LLM, embedding or email failures.	Nginx / cloud load balancer, @nestjs/terminus, managed PostgreSQL
5	Performance	Redis caching of answers, pgvector index for fast similarity search, database indexes and pagination, and timeouts on external calls. Target: standard endpoints under 300 ms and cached answers under 100 ms.	Redis, pgvector (HNSW / IVFFlat index), PostgreSQL indexes

References:

<https://docs.nestjs.com/>

<https://github.com/pgvector/pgvector>

<https://c4model.com/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>